

Rede Bicicleta Semi-Autônoma

Subsistema de Comunicação Sem Fio e Realimentação Multissensorial para Deficientes Visuais

Lara Trabachini, Viktor Miranda e Vinícius César Bertolo Calligaris

Resumo Executivo:

Este documento apresenta a análise técnica detalhada dos 12 arquivos de código-fonte que constituem o projeto *Rede Bicicleta Semi-Autônoma*. O sistema implementa uma rede veicular sem fio ad-hoc baseada em microcontroladores ESP32 e no protocolo de camada de enlace ESP-NOW. Desenvolvido para guiar ciclistas com deficiência visual em uma bicicleta semi-autônoma, o software coordena o envio contínuo de sinais de navegação com realimentação tátil em ambas as manoplas do guidão e comandos de voz via síntese de áudio dedicada (DFPlayer Mini). São detalhados todos os componentes de software, estruturas de dados, máquinas de estado, mecanismos de fail-safe e tolerância a falhas, além do fluxo bidirecional de mensagens entre nós.

Sumário

1	Introdução e Contexto do Projeto	2
1.1	Desafios e Requisitos Críticos de Projeto	2
1.2	Topologia da Rede Embarcada	2
2	Análise Individual dos Arquivos da Pasta Hapticcomm	4
2.1	Config.h — Parâmetros Globais e Seleção de Modo	4
2.2	Message.h — Protocolo da Camada de Aplicação	5
2.3	Audios.h — Tabela Semântica e Mapeamento de Voz	5
2.4	Comms.h — Interface da Camada de Comunicação	6
2.5	Comms.cpp — Implementação do Protocolo ESP-NOW	6
2.6	Hapticcomm.ino — Ponto de Entrada da Aplicação	7
2.7	Transmissor.h — Interface do Nó Mestre	7
2.8	Transmissor.cpp — Lógica de Controle do Mestre e Fail-Safe	7
2.9	Receptor.h — Interface dos Nós Hápticos	8
2.10	Receptor.cpp — Atuação Háptica, Modulação e Resposta de ACK	8
2.11	Espaudio.h — Interface do Nó de Áudio	9
2.12	Espaudio.cpp — Driver Acústico e Supressão de Rajada	9
3	Fluxo de Dados e Interações entre Arquivos e Nós	11
3.1	Árvore de Dependências e Compilação Condicional	11
3.2	Fluxo de Execução Normal e Handshake Periódico	11
3.3	Máquina de Estados de Falha e Comportamento Fail-Safe	12
3.3.1	Cenário de Falha Unilateral ou Bilateral	13
4	Matriz de Configuração e Especificações Técnicas	14
4.1	Parâmetros Temporais e de Desempenho	14
5	Considerações de Engenharia e Recomendações	15
6	Conclusão	15

1 Introdução e Contexto do Projeto

O projeto da **Rede Bicicleta Semi-Autônoma** propõe uma tecnologia assistiva inovadora voltada a devolver a autonomia e a prática esportiva segura para pessoas com deficiência visual (DV). Em veículos convencionais, a percepção sensorial do ambiente baseia-se prioritariamente no canal óptico. Na ausência da visão, faz-se estritamente necessário substituir e complementar o canal visual por canais sensoriais remanescentes de alta fidelidade: o **sistema tátil/háptico** (nas mãos do ciclista) e o **sistema auditivo** (áudio binaural/direcional).

O diretório **Hapticcomm** reúne o conjunto completo de arquivos em C++/Arduino responsáveis pela camada de comunicação sem fio, atuadores de vibração (motores excêntricos acionados via PWM) e geração de comandos sonoros por voz gravada.

1.1 Desafios e Requisitos Críticos de Projeto

A condução de uma bicicleta em velocidades típicas (10 a 25 km/h) impõe restrições severas de latência, confiabilidade e segurança operacional (*safety-critical*):

1. **Latência Ultrabaixa:** Instruções de correção de rota e alertas de obstáculos iminentes devem atingir os atuadores com atraso inferior a dezenas de milissegundos.
2. **Imunidade a Cabeamento Móvel:** O guidão da bicicleta realiza giros contínuos. A passagem de cabos físicos entre a unidade de processamento central (frequentemente alojada no quadro) e as manoplas/áudio sofre com fadiga mecânica, risco de rompimento e limitações ergonômicas. Daí a necessidade de uma rede sem fio local distribuída.
3. **Mecanismo Fail-Safe (Parada Segura):** Se a comunicação com qualquer uma das manoplas for interrompida (por interferência de RF, perda de bateria ou dano físico), o usuário **não pode** continuar pedalando sem aviso. O sistema deve detectar a perda imediatamente (tempo ≤ 300 ms), sinalizar o erro por vibração pulsada de emergência e emitir um áudio explícito localizando a falha.

1.2 Topologia da Rede Embarcada

A arquitetura de hardware distribui quatro microcontroladores ESP32 em pontos estratégicos da bicicleta:

- **Nó Transmissor (Mestre):** Instalado na unidade central de processamento da bicicleta. Lê dados do sistema de percepção (câmeras, LIDAR, GNSS/IMU) e traduz as decisões em pacotes de navegação e alertas.
- **Nó Receptor Direito (Escravo Háptico):** Instalado na manopla direita do guidão. Controla o atuador vibratório direito via PWM.
- **Nó Receptor Esquerdo (Escravo Háptico):** Instalado na manopla esquerda do guidão. Controla o atuador vibratório esquerdo via PWM.
- **Nó Receptor de Áudio (Escravo Acústico):** Conectado ao módulo DFPlayer Mini e autofalante/fones de ouvido, emitindo alertas vocais contextualizados.

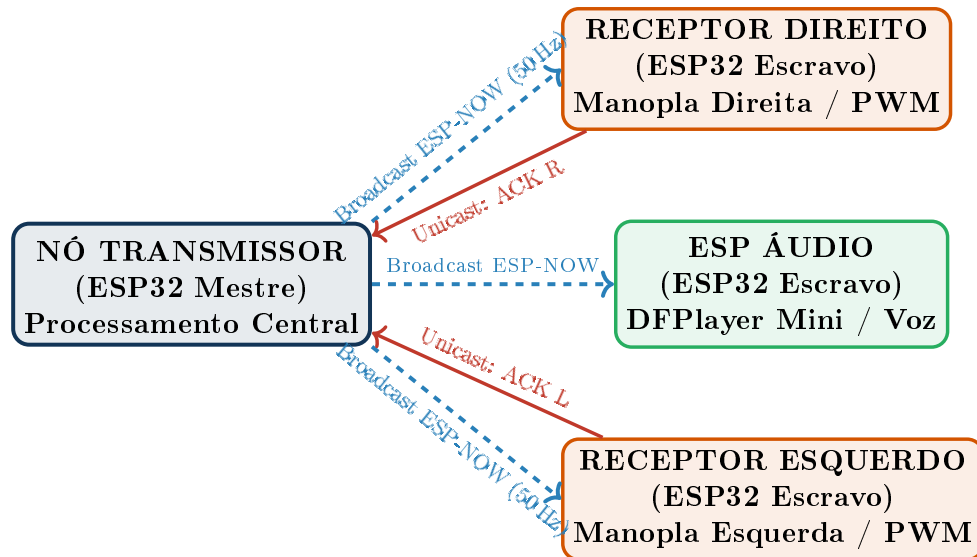


Figura 1: Topologia em estrela da rede sem fio baseada em ESP-NOW com envio por difusão (broadcast) e confirmação seletiva por unicast.

2 Análise Individual dos Arquivos da Pasta Haptic-comm

A biblioteca é estruturada em 12 arquivos modulares em C++ e cabeçalhos (.h, .cpp e .ino). O design de software adota o padrão de **código unificado multi-alvo**: o mesmo repositório atende a todas as 4 placas da bicicleta, onde a definição do papel de cada placa é realizada no cabeçalho de configuração por compilação condicional (*preprocessor macros*).

A seguir, analisa-se cada arquivo em detalhes.

2.1 Config.h — Parâmetros Globais e Seleção de Modo

Finalidade: Atua como a fonte única da verdade (*single source of truth*) para o hardware e a configuração de compilação da rede.

- **Seleção de Papel do Nó:** Através de diretivas de pré-processador exclusivas (linhas 10 a 13), o desenvolvedor descomenta apenas a placa que está sendo gravada:
 - MODO_TRANSMISSOR: Compila a lógica de controle mestre.
 - MODO_RECEPTOR_DIREITO: Compila o nó da manopla direita.
 - MODO_RECEPTOR_ESQUERDO: Compila o nó da manopla esquerda.
 - MODO_ESP_AUDIO: Compila o nó de geração de áudio.
- **Mapeamento de Pinos Físicos:**
 - pinoPWM = 18: Pino de saída PWM ligado ao circuito de potência (MOSFET/Driver) do motor de vibração dos receptores táteis.
 - PINO_LED = 2: LED onboard do transmissor, utilizado como indicador de status e sinalizador de emergência.
 - PINO_DFPLAYER_RX = 16 e PINO_DFPLAYER_TX = 17: Pinos da interface serial UART2 do ESP32 para comunicação com o DFPlayer Mini.
- **Temporizações e Parâmetros da Rede:**
 - INTERVALO_ENVIO = 20 ms: Período de emissão de pacotes pelo transmissor (equivalente a 50 Hz).
 - PACOTES_POR_CICLO = 5: Define a janela de observação. A cada 5 transmissões (100 ms), o transmissor audita se os nós escravos responderam.
 - TIMEOUT_ACK = 300 ms: Limite máximo tolerado sem resposta de confirmação dos receptores antes de declarar queda de link e transitar para estado de emergência (MSG_STOP).
 - PACOTES_PARA_ACK = 5: Contador interno nos receptores táteis. A cada 5 pacotes recebidos com sucesso, o receptor envia uma mensagem de confirmação de volta ao mestre.
 - VOLUME_AUDIO = 25: Ganho configurado para o módulo de áudio (escala de 0 a 30).

2.2 Message.h — Protocolo da Camada de Aplicação

Finalidade: Define a sintaxe e a semântica dos pacotes transmitidos pelo canal sem fio.

O protocolo de mensagens utiliza uma estrutura compacta binária serializada, garantida sem preenchimento de bytes adicionais pela diretiva de compilação `__attribute__((packed))`. O tamanho total do payload é de **exatamente 5 bytes**:

Tabela 1: Estrutura de dados binária serializada (`struct_message`)

Campo	Tipo	Tamanho	Descrição Funcional
tipoMensagem	uint8_t	1 byte	Identificador do estado do protocolo: • 0x00 (MSG_START): Inicialização de rede / reset de falha. • 0x01 (MSG_DADOS): Operação normal de condução. • 0x02 (MSG_STOP): Alerta crítico de desconexão.
intensidadeDireita	uint8_t	1 byte	Ciclo de trabalho (PWM) da manopla direita (0 a 255).
intensidadeEsquerda	uint8_t	1 byte	Ciclo de trabalho (PWM) da manopla esquerda (0 a 255).
pastaAudio	uint8_t	1 byte	Índice numérico da pasta no cartão SD do DFPlayer (0 = sem áudio).
arquivoAudio	uint8_t	1 byte	Índice numérico da faixa de áudio na pasta (0 = sem áudio).

Vantagem Arquitetural: Com apenas 5 bytes de dados úteis, o tempo de transmissão em ar (*airtime*) no rádio 802.11 é minimizado para menos de alguns microssegundos, reduzindo a quase zero a probabilidade de colisões na banda ISM de 2.4 GHz.

2.3 Audios.h — Tabela Semântica e Mapeamento de Voz

Finalidade: Fornece uma camada de abstração entre as mensagens de navegação/segurança e os arquivos de som gravados no cartão micro-SD do DFPlayer Mini.

- **Estrutura AudioRef:** Agrupa um par de inteiros {`pasta`, `arquivo`}.
- **Enumeração AudioID:** Define mnemônicos legíveis organizados por domínios:
 - *Sistema e Conexão (Pasta 1):* Perda de conexão à direita, à esquerda ou em ambos os lados, sistema pronto, bateria fraca.
 - *Navegação (Pasta 2):* Virar à direita, virar à esquerda, seguir em frente, reduzir velocidade, destino atingido.
 - *Deteção de Obstáculos (Pasta 3):* Obstáculo à frente, à direita ou à esquerda.
- **Array Constante TABELA_AUDIOS:** Mapeia diretamente cada membro de `AudioID` para o respectivo diretório e arquivo físico.
- **Macros de Consulta Rápida:** `AUDIO_PASTA(id)` e `AUDIO_ARQUIVO(id)` permitem indexar a tabela em tempo de compilação/execução com eficiência de ponteiro direto.

2.4 Comms.h — Interface da Camada de Comunicação

Finalidade: Declara os protótipos de funções responsáveis pela inicialização da pilha sem fio ESP-NOW e pelas rotinas de manipulação de peers e eventos de recepção.

Contém as seguintes declarações fundamentais:

```
1 bool commsInit();
2 void commsRegistrarPeer(const uint8_t *mac);
3 void OnDataRecv(const esp_now_recv_info_t *recv_info, const uint8_t *
    incomingData, int len);
```

Destaca-se o uso do tipo `esp_now_recv_info_t`, compatível com o core ESP32 v3.x, garantindo acesso direto ao endereço MAC do transmissor (`recv_info->src_addr`).

2.5 Comms.cpp — Implementação do Protocolo ESP-NOW

Finalidade: Implementa as funções declaradas em `Comms.h`, configurando o hardware Wi-Fi no modo Station (`WIFI_STA`) e orquestrando o roteamento de pacotes recebidos.

- **Definição do Endereço de Broadcast:** Instancia `broadcastAddress = {0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF}`, permitindo que o transmissor alcance todos os nós simultaneamente sem necessidade de pareamento prévio em cada envio.
- **Função `commsInit()`:**
 1. Coloca o Wi-Fi em modo Station (desligando o modo Access Point para economizar energia e evitar interferências).
 2. Inicializa a biblioteca ESP-NOW (`esp_now_init()`).
 3. Registra o endereço de difusão geral (Broadcast) na tabela interna de peers da ESP32 com canal 0 e sem criptografia.
 4. Registra a função de callback `OnDataRecv`.
- **Registro Dinâmico de Peers (`commsRegistrarPeer`):** Verifica se o MAC da estação remota já existe na tabela de peers (`esp_now_is_peer_exist`). Caso não exista, adiciona o MAC como peer unicast. Isso permite que um receptor responda diretamente ao transmissor sem que seu endereço MAC precisasse ser fixado em código (*hardcoded*).
- **Despachante Polimórfico (`OnDataRecv`):** O callback universal inspeciona as macros de compilação e despacha os dados brutos recebidos para o tratador correto do papel específico da placa compilada:
 - Se `MODU_TRANSMISSOR`: Invoca `transmissorProcessarACK(mac, incomingData, len)`.
 - Se `MODU_ESP_AUDIO`: Invoca `espAudioProcessarPacote(mac, incomingData, len)`.
 - Caso contrário (Receptores Táteis): Invoca `receptorProcessarPacote(mac, incomingData, len)`.

2.6 Hapticcomm.ino — Ponto de Entrada da Aplicação

Finalidade: Constitui o arquivo principal da aplicação no ecossistema Arduino/ESP32, contendo as funções `setup()` e `loop()`.

- **Rotina `setup()`:**

1. Inicializa a comunicação serial com taxa de 115200 bauds para depuração.
2. Aplica um atraso (`delay(1000)`) para estabilização de hardware e alimentação.
3. Inicializa a pilha de comunicação invocando `commsInit()`.
4. Redireciona a inicialização de hardware específica via macros para `transmissorSetup()` ou `receptorSetup()`.

- **Rotina `loop()`:** Encaminha o ciclo de varredura contínuo para `transmissorLoop()` ou `receptorLoop()` (o nó de áudio é atendido por interrupção de recepção).

2.7 Transmissor.h — Interface do Nó Mestre

Finalidade: Declara os protótipos das funções que regem o comportamento da unidade transmissora mestre:

```
1 void transmissorSetup();  
2 void transmissorLoop();  
3 void transmissorProcessarACK(const uint8_t *mac, const uint8_t *  
    incomingData, int len);
```

Todo o arquivo é cercado pela proteção condicional `#ifdef MODO_TRANSMISSOR`.

2.8 Transmissor.cpp — Lógica de Controle do Mestre e Fail-Safe

Finalidade: Implementa o cérebro da rede de comunicação, sendo responsável pela temporização dos envios, auditoria de conexão, máquina de estados de falha e recuperação automática.

- **Variáveis de Estado:**

- `ackRecebidoR`, `ackRecebidoL`: Marcadores booleanos que registram a confirmação de recebimento pelos lados direito e esquerdo.
- `erroConexao`, `falhaR`, `falhaL`: Flags que indicam falha geral ou falha específica em um dos rádios periféricos.
- `pacotesEnviados`: Contador da janela móvel de transmissão.

- **Procedimento `enviarStart()`:** Transmite um pacote de controle `MSG_START` com intensidades e áudios zerados, utilizado tanto na energização da bicicleta quanto no instante de recuperação de uma falha prévia.

- **Tratamento de Confirmações (`transmissorProcessarACK`):** Converte os dados recebidos em string e verifica o conteúdo:

- Se "ACK R": Marca `ackRecebidoR = true`.
- Se "ACK L": Marca `ackRecebidoL = true`.

- **Ciclo Contínuo (transmissorLoop):**

1. **Temporizador de Envio (20 ms):** Emissão em alta frequência de mensagens no canal de broadcast. Em modo normal, envia MSG_DADOS com PWM nominal (ex.: 128 / 50%). Em modo de erro (`erroConexao = true`), força MSG_STOP com PWM de pulso (200) e codifica o áudio de perda correspondente (AUDIO_PERDA_DIREITA, AUDIO_PERDA_ESQUERDA ou AUDIO_PERDA_AMBOS).
2. **Auditoria de Conexão por Janela:** A cada 5 pacotes transmitidos (100 ms), inicia a contagem de timeout (`tempoUltimoEnvio = millis()`) e zera os flags de ACK.
3. **Avaliação de Timeout (300 ms):** Caso transcorram mais de 300 ms sem a chegada de ambos os ACKs, o mestre ativa `erroConexao = true`, identificando exatamente qual manopla falhou. Se a conexão for restabelecida no ciclo seguinte, o erro é cancelado e `enviarStart()` é invocado para reativar o sistema.
4. **Sinalização Visual de Emergência:** Em caso de erro, o LED onboard (pino 2) pisca em frequência rápida de 2.5 Hz (período de alternância de 200 ms).

2.9 Receptor.h — Interface dos Nós Hápticos

Finalidade: Declara os protótipos de rotina para os nós atuadores instalados no guidão da bicicleta:

```
1 void receptorSetup();
2 void receptorLoop();
3 void receptorProcessarPacote(const uint8_t *mac, const uint8_t *
    incomingData, int len);
```

O cabeçalho é ativo somente quando não é nem transmissor nem placa de áudio (`!MODO_TRANSMISSOR && !MODO_ESP_AUDIO`).

2.10 Receptor.cpp — Atuação Háptica, Modulação e Resposta de ACK

Finalidade: Recebe as instruções de rádio, aplica a modulação de largura de pulso (PWM) no motor de vibração e monitora a periodicidade de recebimento para devolver o sinal de confirmação (ACK) ao mestre.

- **Inicialização (receptorSetup):** Configura o pino 18 (`pinoPWM`) como saída digital modulada e garante nível zero na inicialização.
- **Filtragem e Decodificação (receptorProcessarPacote):**
 - Valida o comprimento do pacote recebido (`len == sizeof(struct_message)`).
 - Copia o pacote para a estrutura global `data`.
 - **Filtragem de Canal:** Conforme a macro de compilação (`MODO_RECEPTOR_DIREITO` ou `MODO_RECEPTOR_ESQUERDO`), o nó extrai apenas `data.intensidadeDireita` ou `data.intensidadeEsquerda`, descartando dados de áudio e do lado oposto.
- **Processamento dos Tipos de Mensagem:**

- **MSG_START:** Cancela o modo de pulso e desliga a vibração.
 - **MSG_DADOS:** Aplica a intensidade correspondente no motor via `analogWrite(pinoPWM, minhaIntensidade)`.
 - **MSG_STOP:** Entra no estado de alerta `modoPulso = true`, fixando a intensidade em 200 para vibração intermitente.
- **Geração de Pulso (receptorLoop):** Quando em `modoPulso`, o loop alterna a saída PWM entre ligado (intensidade 200) e desligado (0) a cada 250 ms (efeito estroboscópico tátil), indicando alerta tátil inconfundível para o ciclista deficiente visual.
 - **Envio Periódico de ACK:** Incrementa `contadorPacotes`. A cada 5 pacotes recebidos (`PACOTES_PARA_ACK = 5`), registra o endereço MAC do transmissor e despacha uma mensagem unicast com o texto "ACK R" (se receptor direito) ou "ACK L" (se receptor esquerdo).

2.11 Espaudio.h — Interface do Nó de Áudio

Finalidade: Declara os métodos de inicialização, ciclo de monitoramento e tratamento de pacotes dedicados ao módulo acústico:

```

1 void espAudioSetup();
2 void espAudioLoop();
3 void espAudioProcessarPacote(const uint8_t *mac, const uint8_t *
    incomingData, int len);

```

Condicionado exclusivamente à macro `MODULO_ESP_AUDIO`.

2.12 Espaudio.cpp — Driver Acústico e Supressão de Rajada

Finalidade: Faz a interface física com o módulo reproduzidor de MP3/WAV DFPlayer Mini via UART secundária da ESP32 e implementa um filtro temporal (*anti-retriggering debounce*) indispensável para evitar engasgos de som causados pela alta taxa de repetição do transmissor.

- **Comunicação UART com DFPlayer Mini:** Utiliza a instância `HardwareSerial dfSerial(2)` mapeada nos pinos 16 (RX da ESP32 ← TX do módulo) e 17 (TX da ESP32 → RX do módulo) a 9600 bps.
- **O Problema da Rajada de Pacotes:** Como o transmissor emite mensagens a 50 Hz (a cada 20 ms), enviar repetidamente a ordem de tocar o mesmo áudio causaria interrupções milissegundo a milissegundo no reproduzidor, gerando ruído ininteligível.
- **Mecanismo de Filtro de Rajada (tocar):**

```

1 bool mesmoAudio = (pasta == ultimaPasta && arquivo == ultimoArquivo);
2 if (mesmoAudio && (millis() - tempoUltimoAudio <
    INTERVALO_MIN_AUDIO)) return;

```

A reprodução da mesma faixa só é reenviada ao hardware após um intervalo mínimo de 800 ms (`INTERVALO_MIN_AUDIO`), garantindo que frases como "Vire à direita" ou "Obstáculo à frente" sejam reproduzidas de maneira contínua e compreensível.

- **Tratamento de Mensagens:**

- **MSG_START:** Reseta as variáveis `ultimaPasta` e `ultimoArquivo` para zero, preparando o canal acústico para novas emissões.
- **MSG_DADOS** e **MSG_STOP:** Invoca a função `tocar(data.pastaAudio, data.arquivoAudio)`. Se os valores forem nulos (zero), nenhum som é emitido.

3 Fluxo de Dados e Interações entre Arquivos e Nós

A integração dos arquivos obedece a duas dimensões fundamentais:

1. **Dimensão de Compilação (Estrutural):** Como os arquivos são incluídos e mesclados em tempo de compilação dependendo da placa-alvo.
2. **Dimensão de Execução (Dinâmica):** Como as mensagens trafegam pelo ar, como os eventos assíncronos disparam callbacks e como as máquinas de estado reagem a falhas.

3.1 Árvore de Dependências e Compilação Condicional

A Figura 2 demonstra a hierarquia de inclusão dos arquivos de cabeçalho e implementação no processo de compilação.

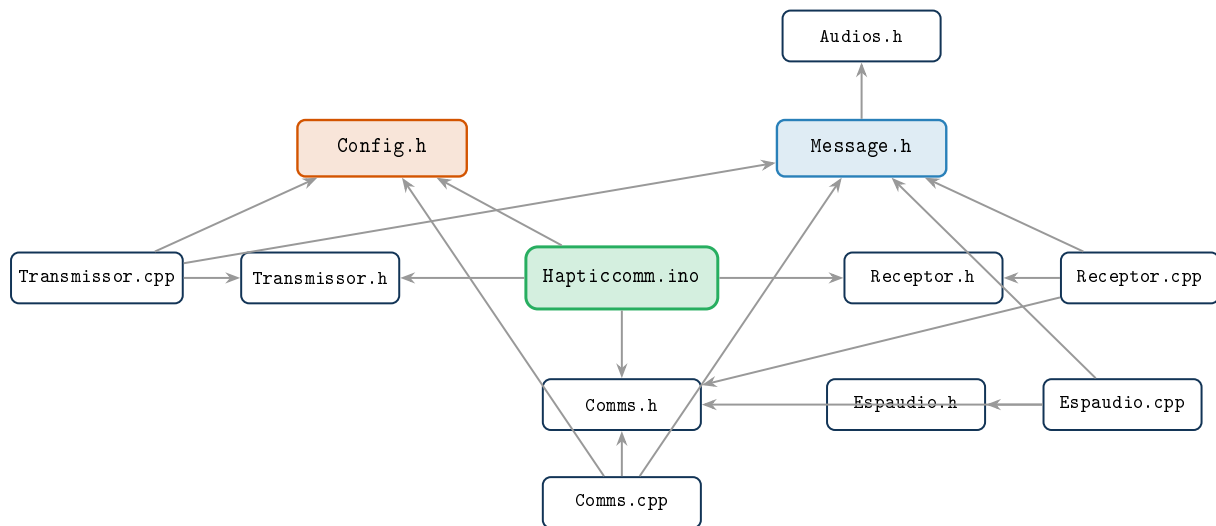


Figura 2: Grafo de dependências e inclusões entre os arquivos do projeto Rede Bicicleta Semi-Autônoma.

3.2 Fluxo de Execução Normal e Handshake Periódico

Durante o funcionamento nominal sem falhas, o sistema opera sob regime contínuo de broadcast e confirmação por janela temporal.

1. **Fase de Inicialização (setup):** O transmissor executa `enviarStart()`, disparando um pacote com `tipoMensagem = MSG_START`. Todos os receptores que recebem este pacote resetam seus estados de alarme e preparam seus buffers.
2. **Fase de Operação Normal (Ciclo de 20 ms):** A cada 20 ms, o mestre monta uma mensagem `MSG_DADOS`, preenche as intensidades de PWM para cada lado e o código do áudio que deve ser executado no momento, e envia para o endereço de broadcast (`FF:FF:FF:FF:FF:FF`).
3. **Recepção e Processamento Local:**

- O **Receptor Direito** filtra `intensidadeDireita` e atualiza o ciclo de trabalho do pino 18.

- O **Receptor Esquerdo** filtra `intensidadeEsquerda` e atualiza seu respectivo pino 18.
 - A **ESP de Áudio** decodifica `pastaAudio` e `arquivoAudio`, acionando o DF-Player apenas se os índices forem válidos e o tempo decorrido desde o último áudio idêntico superar 800 ms.
4. **Auditoria de Integridade (ACK):** Os receptores táteis mantêm um contador. Ao acumularem 5 pacotes recebidos, despacham um frame unicast de confirmação ("ACK R" ou "ACK L"). O mestre recebe essas respostas e marca as flags `ackRecebidoR` e `ackRecebidoL`.

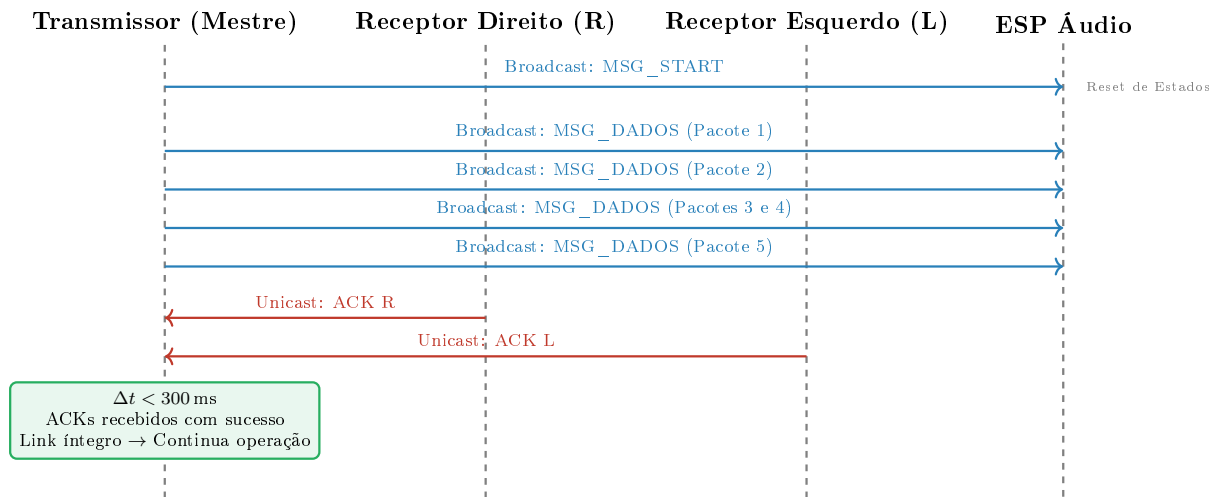


Figura 3: Diagrama de sequência da operação normal com transmissão periódica e confirmações por janela.

3.3 Máquina de Estados de Falha e Comportamento Fail-Safe

A segurança de um usuário deficiente visual em deslocamento rápido exige uma reação determinística quando um enlace sem fio é interrompido. A Figura 4 ilustra a máquina de estados implementada no nó transmissor e sua correspondência nos nós receptores.

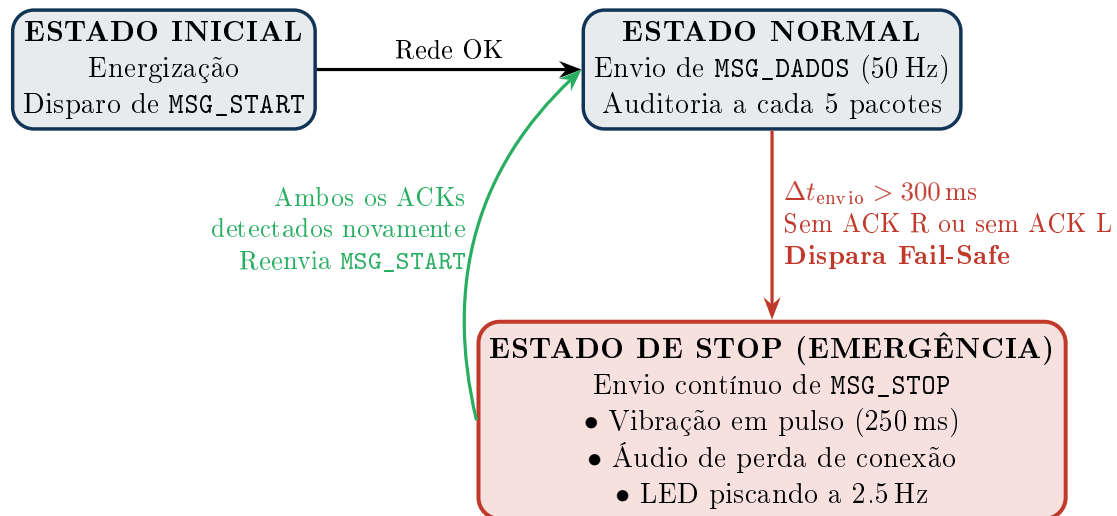


Figura 4: Máquina de estados finitos do nó transmissor mestre com transições de proteção.

3.3.1 Cenário de Falha Unilateral ou Bilateral

Quando o temporizador `millis() - tempoUltimoEnvio > TIMEOUT_ACK` expira sem que as variáveis `ackRecebidoR` ou `ackRecebidoL` tenham sido validadas, a lógica em `Transmissor.cpp` ativa a rotina de contenção:

1. Identifica se a falha é exclusiva da manopla direita (`falhaR`), exclusiva da manopla esquerda (`falhaL`) ou mútua (`falhaR && falhaL`).
2. Seleciona o áudio correspondente na tabela:
 - `AUDIO_PERDA_DIREITA` (Pasta 1, Arquivo 1): Informa ao usuário que a manopla direita perdeu sinal.
 - `AUDIO_PERDA_ESQUERDA` (Pasta 1, Arquivo 2): Informa falha na manopla esquerda.
 - `AUDIO_PERDA_AMBOS` (Pasta 1, Arquivo 3): Informa falha geral de conexão.
3. Comuta as variáveis de transmissão para `tipoMensagem = MSG_STOP` e intensidades em 200.
4. Nos receptores táteis que ainda estiverem ao alcance, a chegada do comando `MSG_STOP` aciona a rotina interna `modoPulso = true`. O `receptorLoop()` passa a chavear o motor de vibração a cada 250 ms (2Hz), criando um aviso tátil inconfundível de alerta para que o ciclista pare a bicicleta imediatamente.
5. Caso a conexão se restabeleça, o transmissor detecta a volta dos ACKs, reseta os sinalizadores de erro e retransmite `MSG_START`, restaurando a operação regular de condução de forma suave.

4 Matriz de Configuração e Especificações Técnicas

A Tabela 2 consolida as diferenças operacionais de cada nó da bicicleta com base nas macros ativadas em `Config.h`.

Tabela 2: Matriz comparativa de funções, hardware e comportamento dos nós

Nó / Macro	Pinos de I/O	Mensagens Emi- tidas	Comportamento em Recep- ção
Transmissor (<code>MODO_TRANSMISSOR</code>)	Pino 2 (LED status) USB-Serial (debug/host)	Broadcast: • <code>MSG_START</code> • <code>MSG_DADOS</code> • <code>MSG_STOP</code>	Recebe unicast "ACK R" e "ACK L". Gerencia timeout de 300 ms e chaveamento de estados de falha.
Receptor Direito (<code>MODO_RECEPTOR_DIREITO</code>)	Pino 18 (PWM Motor Direito)	Unicast: • "ACK R" (a cada 5 pacotes)	Filtra <code>intensidadeDireita</code> . Se <code>MSG_DADOS</code> , aciona PWM direto. Se <code>MSG_STOP</code> , pulsa motor a cada 250 ms.
Receptor Esquerdo (<code>MODO_RECEPTOR_ESQUERDO</code>)	Pino 18 (PWM Motor Esquerdo)	Unicast: • "ACK L" (a cada 5 pacotes)	Filtra <code>intensidadeEsquerda</code> . Se <code>MSG_DADOS</code> , aciona PWM direto. Se <code>MSG_STOP</code> , pulsa motor a cada 250 ms.
ESP Áudio (<code>MODO_ESP_AUDIO</code>)	Pino 16 (RX2 ESP) Pino 17 (TX2 ESP)	Nenhuma (nó passivo de escuta)	Decodifica <code>pastaAudio</code> e <code>arquivoAudio</code> . Aplica filtro de repetição de 800 ms e comanda o DFPlayer Mini via UART.

4.1 Parâmetros Temporais e de Desempenho

Os parâmetros temporais da rede foram afinados experimentalmente para assegurar tempo de resposta compatível com a dinâmica veicular de uma bicicleta:

- **Frequência de Atualização do Mestre:** 50 Hz (período de 20 ms).
- **Janela de Confirmação (ACK):** A cada 5 pacotes recebidos (100 ms de intervalo nominal).
- **Tempo Limite para Detecção de Queda de Link (*Timeout*):** 300 ms (equivalente à perda consecutiva de 3 janelas de confirmação, mitigando falsos positivos decorrentes de ruído passageiro).
- **Frequência de Pulso no Estado de Parada:** 2 Hz (250 ms ligado, 250 ms desligado).
- **Filtro Anti-Retrigger de Mensagens Vocais:** 800 ms.

5 Considerações de Engenharia e Recomendações

A análise aprofundada dos arquivos da pasta `Hapticcomm` demonstra que a arquitetura foi desenhada com alto grau de maturidade, priorizando a segurança física do usuário deficiente visual através de redundância de canais (vibração + voz) e verificação contínua de integridade de enlace.

Com base nos princípios de engenharia de software embarcado e segurança funcional, destacam-se algumas oportunidades de evolução técnica:

1. **Inclusão de Checksum ou CRC no Payload:** Atualmente, a estrutura `struct_message` possui 5 bytes sem campo de integridade de aplicação. Embora o protocolo de rádio 802.11 e o ESP-NOW apliquem CRC-32 na camada de enlace (descartando quadros corrompidos em hardware), adicionar um contador de sequência (*sequence counter*) ou byte de paridade no nível de aplicação permite aos receptores mensurar a taxa real de perda de pacotes (*Packet Loss Rate*).
2. **Confirmação da ESP de Áudio:** Atualmente, os nós que respondem com confirmação (ACK) são os receptores táteis direito e esquerdo. O nó de áudio atua em modo passivo. Caso a segurança exija garantir que o canal de voz também esteja operante, pode-se incluir uma confirmação periódica da ESP de áudio ("ACK A"), assegurando redundância auditiva total.
3. **Priorização de Mensagens Críticas no DFPlayer Mini:** Em manobras urgentes (ex.: frenagem imediata por obstáculo súbito à frente), o comando de áudio pode precisar interromper um áudio descritivo longo que esteja em execução. Uma instrução explícita de *Stop/Abort* antes da reprodução da faixa de emergência pode ser adicionada no módulo `Espaudio.cpp`.

6 Conclusão

O projeto **Rede Bicicleta Semi-Autônoma** provê a infraestrutura de comunicação essencial para a condução segura por ciclistas com deficiência visual. A arquitetura em rede local ad-hoc utilizando ESP-NOW supera as fragilidades e custos de cabeamento físico em guidão rotativo, mantendo latências na faixa de unidades de milissegundos.

A segregação do código em 12 módulos interdependentes e altamente parametrizáveis permite a compilação de quatro perfis de nós a partir de uma base comum, facilitando manutenção e testes de bancada. O mecanismo de *fail-safe* implementado assegura que qualquer descontinuidade no enlace de rádio seja imediatamente percebida pelo usuário através de vibração intermitente e avisos de voz específicos, garantindo os padrões de segurança necessários para uma tecnologia assistiva veicular de alto impacto social.